Практическая работа №5

Тема:

Полнотекстовый поиск и интеграция OpenSearch

Цель: настроить индексацию сущности «Документ», реализовать механизм синхронизации данных между PostgreSQL и OpenSearch, а также создать интерфейс полнотекстового поиска.

Краткие теоретические сведения

OpenSearch - это распределенная поисковая и аналитическая система с открытым исходным кодом. Она позволяет выполнять сложный поиск по тексту, учитывая морфологию слов, синонимы и релевантность.

Инвертированный индекс - основной механизм поиска. Вместо того чтобы просматривать каждый документ целиком, система создает словарь всех слов и ссылок на документы, где они встречаются. Это делает поиск практически мгновенным.

Маппинг (Mapping) - процесс определения структуры документа в индексе (аналог схемы таблицы в БД). Здесь задается, какие поля будут текстовыми (для поиска), а какие — ключевыми (для фильтрации).

MultiMatchQuery - запрос, который ищет совпадения одновременно по нескольким полям. Поддерживает **boosting** - настройку веса полей: например, совпадения в заголовке могут считаться важнее, чем в содержании

Задание:

ФИО студента должно быть явно видно на каждом этапе:

В терминале (Докер и OpenSearch) - перед запуском контейнеров выведите свое ФИО в консоль (например, командой echo "Студент: Иванов Иван Иванович"), чтобы оно попало на скриншоты терминала.

В базе данных (тестовые данные)-в тексте content хотя бы одного документа должно быть указано ваше ФИО (например: «Настоящий приказ

подготовлен специалистом отдела кадров Ивановым И.И.»). Это же ФИО должно отображаться в результатах поиска на скриншотах UI.

Часть 1. Инфраструктура и создание индекса

Убедитесь, что в вашем docker-compose.yml запущен контейнер OpenSearch и доступен по порту 9200 (если его нет - добавьте, отключив плагин безопасности для локальной разработки). Проверьте, что контейнер запущен (docker compose ps, добавьте скриншот в отчет) и OpenSearch отвечает на GET http://localhost:9200 - в ответе должен быть JSON с версией и полем tagline (добавьте скриншот в отчет).

С помощью REST-клиента (Postman или curl) создайте индекс documents_index. При создании укажите number_of_shards = 1 и number_of_replicas = 0 - иначе на одноузловом кластере индекс будет в статусе yellow (реплику некуда разместить).

Настройте маппинг для полей: *title (text), reg_number (keyword), content (text)*. Для полей *title* и *content* укажите анализатор russian - это обеспечит морфологический поиск по русскому языку (стемминг, стоп-слова). Без него поиск по корню слова («приказ» -> «приказами») работать не будет.

Проверьте результат: GET /_cat/indices?v (индекс documents_index должен появиться) и GET /documents_index/_mapping (типы полей и анализатор должны соответствовать заданным). Предоставьте скриншоты.

Примечание: *если вы используете Spring Data OpenSearch, индекс может быть создан автоматически при первом запуске приложения по аннотации @Document. В этом случае проверьте маппинг через GET /documents_index/_mapping; при необходимости удалите (DELETE) и пересоздайте индекс вручную с нужной конфигурацией.*

Часть 2. Синхронизация данных

В приложении Java настройте клиент для работы с OpenSearch (используя opensearch-java или Spring Data OpenSearch). Выберите один из

двух подходов: (а) низкоуровневый клиент opensearch-java - прямая работа с REST API; (б) высокоуровневый Spring Data OpenSearch - репозитории по аналогии с JPA. Рекомендуется второй подход. При использовании Spring Data OpenSearch версия зависимости должна соответствовать версии Spring Boot - сверяйтесь с матрицей совместимости проекта(Spring Boot 4.1.x требует spring-data-opensearch 3.1.x). При несовпадении версий приложение не запустится или будет работать некорректно.

Укажите адрес OpenSearch в *application.properties* (для Spring Data OpenSearch - свойство *spring.data.opensearch.uris*). Если возникают ошибки при использовании Spring Data OpenSearch исключите автоконфигурацию Elasticsearch в главном классе приложения, чтобы избежать конфликта клиентов.

Реализуйте сервис индексации: при сохранении нового документа в базу через DocumentService, его копия в упрощенном виде (DTO) должна отправляться в OpenSearch. Создайте отдельный DTO-класс для индекса.

Добавьте в сущность Document поле content (тип TEXT).

Если проиндексировать документ до сохранения в БД, его id будет null. OpenSearch сгенерирует случайный идентификатор, и при повторной индексации того же документа появится дубль. Поэтому индексируйте объект, возвращенный repository.save(...) - у него уже есть реальный ID из БД.

Подготовка тестовых данных (обязательно перед тестированием поиска)

Для проверки полнотекстового поиска (Части 3 и 4) в базе должно быть не менее 5-7 документов с осмысленным текстом в поле «content».

Требования к тестовым данным

Объем текста

У каждого документа «content» должен содержать связный текст не менее 3-5 предложений (примерно 200–500 символов). Короткая строка вроде

«тест» или «текст документа» не подходит - в ней нет «середины», негде проверять морфологию.

Искомое слово - в середине текста

Хотя бы в одном документе слово, по которому будет демонстрироваться поиск в отчете (п. 3), должно стоять не в начале и не в заголовке, а в середине текста.

Морфологические формы

Подготовьте документы, где одно и то же слово встречается в разных формах: «приказ», «приказа», «приказами», «приказы». Это нужно для сравнения поиска по корню и по точной форме.

Boosting (title, content)

Хотя бы один документ должен содержать искомое слово и в заголовке, и в тексте - чтобы при поиске с весом заголовка он оказался выше документа, где это слово только в тексте. Это визуальное подтверждение работы boosting.

Способы заполнения (на выбор студента):

- через UI-форму создания документа (из ПР №3) - но нужно добавить в форму многострочное поле для *content*;

- через SQL-вставку прямо в PostgreSQL - несколько строк с реалистичным текстом;

- через CommandLineRunner / DataInitializer в Java-коде - заполняет БД при старте, если таблица пуста.

Часть 3. Реализация полнотекстового поиска

В DocumentService добавьте метод search(String query), который формирует MultiMatchQuery к OpenSearch. Запрос должен искать по полям title и content одновременно.

Перед реализацией на Java проверьте работу поиска напрямую через REST: ‘POST /documents_index/_search’ с запросом multi_match по полям title

и content. Убедитесь, что результаты релевантны, и только затем переносите логику в Java-код.

Настройте параметры поиска так, чтобы совпадения в заголовке имели больший «вес» (boosting), чем в тексте. Используйте синтаксис поля с весом: например, title^3 - совпадения в заголовке считаются в 3 раза важнее. Конкретное значение веса (2, 3 или 5) - на усмотрение студента, но оно должно быть задано явно. Метод search должен возвращать ограниченный набор результатов (например, первые 50) - иначе при большом количестве совпадений UI будет работать медленно.

Часть 4. Разработка UI-компонента поиска

На странице списка документов (из ПР №3) добавьте расширенную панель поиска. Панель разместите над Grid; компоненты - текстовое поле (TextField) для ввода запроса и, при необходимости, кнопка «Найти».

Вместо простого фильтра JPA подключите вызов метода поиска через OpenSearch. При пустом запросе показывайте все документы из БД; при непустом - результаты OpenSearch. Для отображения результатов используйте тот же Grid (с типом элементов - DTO индекса) или отдельный Grid.

Реализуйте подсветку (highlighting) найденных фрагментов текста в результатах поиска (по желанию). Для этого в запросе к OpenSearch добавьте секцию highlight для поля content и извлеките подсвеченные фрагменты из результатов, отобразив их в компоненте HTML.

Содержание отчета

1. **Часть 1**
    a. Скриншот терминала, работающие контейнеры (docker compose ps), ответ curl http://localhost:9200.
    b. Скриншот создания индекса и проверки маппинга (GET /_mapping).

c. *Листинг/скриншот docker-compose файла*

2. **Часть 2**
    a. Скриншоты кода: зависимость в pom.xml, настройки в application.properties, классы Document (с полем content) и DocumentIndexService.
    b. Скриншот UI: не менее 5-7 документов с текстом. *В тексте хотя бы одного документа должно быть указано ФИО.*
3. **Часть 3**
    a. Скриншот REST-запроса (POST /_search).
    b. Скриншот Java-метода search().
4. **Часть 4**
    a. Скриншот кода панели поиска в Vaadin.
    b. Скриншоты интерфейса: поиск слова из середины текста (с ФИО), поиск по корню (морфология) и демонстрация boosting (заголовок выше текста).

Контрольные вопросы:

1. Почему поиск через SQL-оператор **LIKE '%слово%'** неэффективен для больших объемов данных?
2. Что такое инвертированный индекс и из каких двух основных частей он состоит?
3. Чем отличаются типы полей **text** и **keyword** в маппинге OpenSearch? Для каких полей в вашей работе (title, content, reg_number) применен каждый тип и почему?
4. Для чего в практической работе при создании индекса указывается **number_of_replicas = 0**? Что произойдет со статусом кластера, если оставить **1** на одном узле?
5. Что делает анализатор **russian** и какие три ключевых токен-фильтра он использует в своей цепочке?

6. В чем разница между стеммингом и лемматизацией? Какой из этих подходов используется во встроенном анализаторе **russian**?
7. Для чего нужен **MultiMatchQuery** и как в нем реализуется boosting (например, синтаксис **title^3**)?
8. Почему в коде Java индексацию документа в OpenSearch нужно вызывать ***после*** **repository.save()**, а не до него? Что произойдет, если проиндексировать объект с **null** в поле ID?
9. Зачем в Spring Data OpenSearch используется аннотация **@Document** и чем она отличается от JPA-аннотации **@Entity**?
10. Объясните разницу между контекстом **Query** и **Filter** в языке запросов OpenSearch. Что из них кэшируется?
11. Как работает алгоритм ранжирования BM25? Какие три основных фактора он учитывает при вычислении релевантности документа?
12. Почему подход **Dual Writes** (одновременная запись в PostgreSQL и OpenSearch из кода) считается ненадежным? Какую технологию (CDC) используют в production для синхронизации?
13. Что такое шардирование и репликация в OpenSearch? Чем отличается Primary-шард от Replica-шарда?
14. Как в практической работе реализована подсветка (highlighting) найденных фрагментов? В каком разделе JSON-запроса она настраивается?
15. В каких случаях НЕ стоит использовать OpenSearch (назовите хотя бы две причины) и когда лучше ограничиться возможностями реляционной БД?

Список полезных источников:

1. Официальная документация OpenSearch, «Create Index API» - https://docs.opensearch.org/latest/api-reference/index-apis/create-index/

2. Baeldung, «Guide to Elasticsearch in Java» - https://www.baeldung.com/elasticsearch-java